

CAPSTONE DESIGN
INNOVATION | COLLABORATION

Capstone Design

Financial helper chatbot

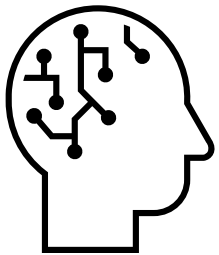
College of Computing



Introduction

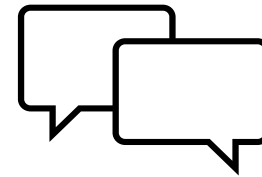
General Purpose LLMs

- » Flexible answer
- » Hallucination problem
- » Difficult to reflect up-to-date information with pre-trained data
- » Allowing web search cannot guarantee reliability

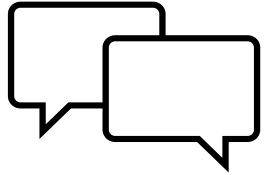


Conventional Chatbots (Bank, etc.)

- » Provide reliable prepared answer
- » Cannot answer for not prepared question
- » High cost to prepare every response
- » New response need to written for each up-to-date information



Introduction



Verifiable Accuracy

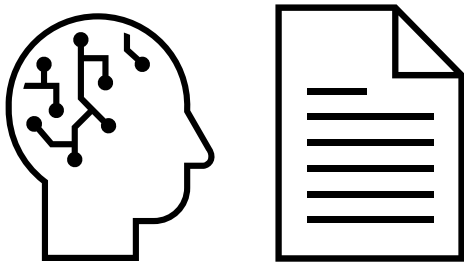
Source Transparency

Accessibility

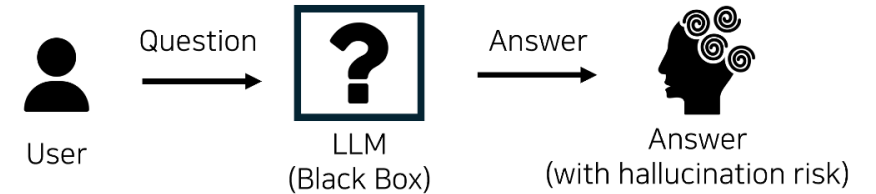
Project Idea

RAG (Retrieval-Augmented Generation)

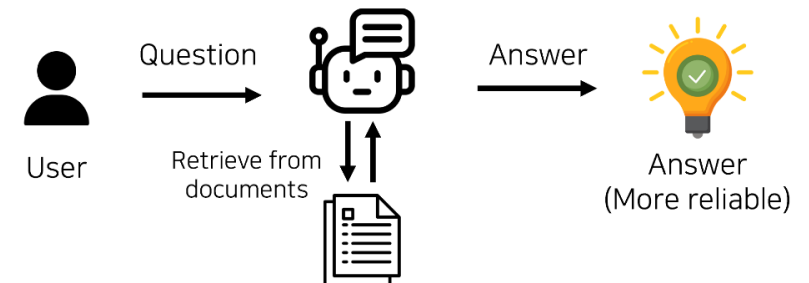
- » Prepare vector store for specialized documents
- » Retrieve similar documents
- » Provide to LLM
- » Flexible answer
- » Easy to add up-to-date information
- » Can track source



General LLM



Our Chatbot (with RAG)



Project Idea

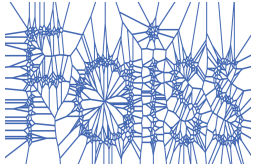
» Frontend interface:



» RAG backend:



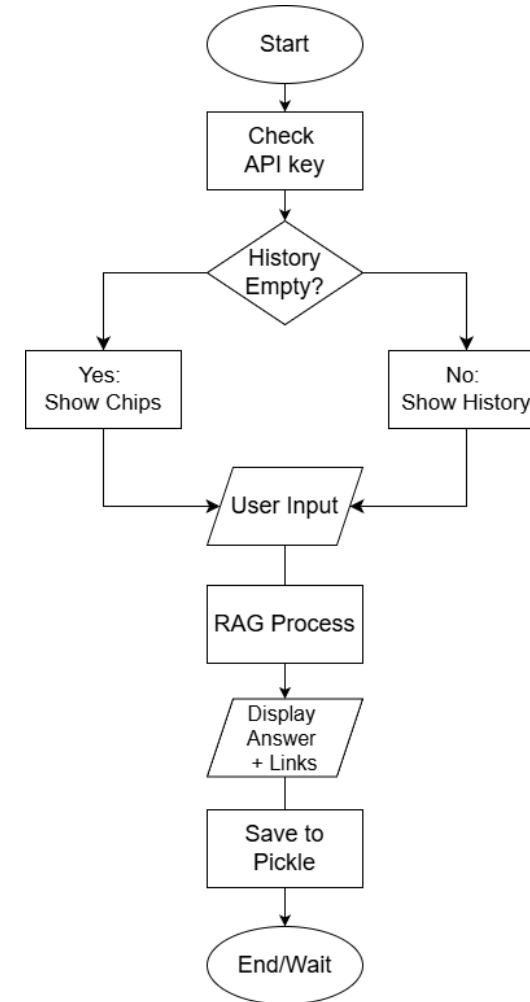
» Vector store:



» History saving:



» LLM, vector embedding: **OpenAI**



Project Idea

Streamlit

- » Python framework to develop, deploy interactive, shareable web service
- » User-friendly interface
- » Fast development
- » Simple interaction handling



Project Idea

LangChain

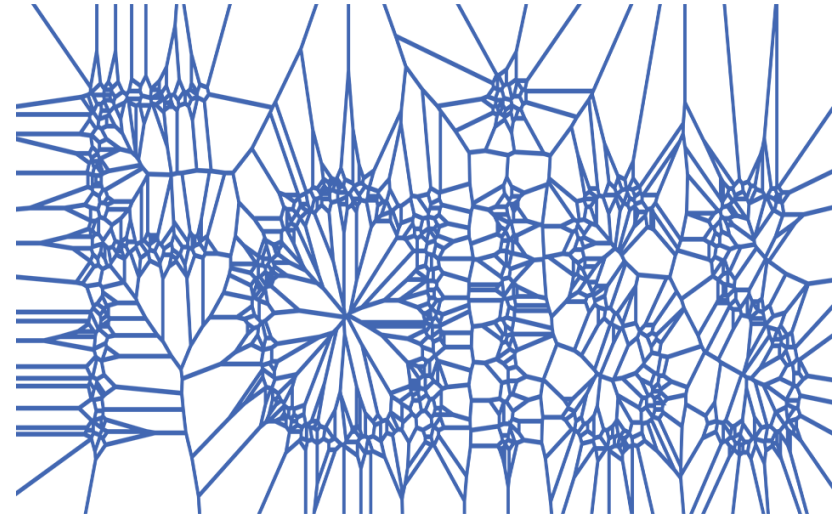
- » Python-based framework for production LLM
- » I/O handling for LLM
- » Compatible data connection with modular design
- » Manage data flow for RAG



Project Idea

FAISS (Facebook AI Similarity Search)

- » Library for similarity search of large data
- » Documents embedded to vector with OpenAI embedding model
- » Embedded vectors indexed and stored by FAISS for efficient search
- » Optimized for memory usage and speed
- » State-of-the-art GPU implementation



Project Idea

Pickle

- » Python module to store data
- » Serialize and store python object directly
- » Enough to implement single-user history saving

Pickle

Project implementation details - Frontend

```
def main():  
    init_chat_state()  
    render_sidebar()  
  
    st.markdown("## 🧑‍💻 Financial Helper Chatbot")  
  
    render_chat_messages()  
    handle_user_input()
```

```
def render_sidebar():  
    with st.sidebar:  
        st.markdown("## ⚙️ Settings")  
  
        default_key = ""  
        try:  
            default_key = st.secrets.get("openai_api_key", "")  
        except Exception:  
            pass  
        if not default_key:  
            default_key = os.getenv("OPENAI_API_KEY", "")  
        if "api_key" not in st.session_state:  
            st.session_state.api_key = default_key  
  
        is_expanded = not bool(st.session_state.api_key)  
        with st.expander("🔑 API Key 설정", expanded=is_expanded):  
            with st.form("api-key-form", clear_on_submit=False):  
                api_key_input = st.text_input("API Key 입력", type="password", value=st.session_state.api_key)  
                if st.form_submit_button("저장"):  
                    st.session_state.api_key = api_key_input.strip()  
                    st.rerun()  
  
        st.markdown("### 📄 대화 기록")  
  
        if st.button("➕ 새 대화 시작", use_container_width=True, type="primary"):  
            start_new_chat()  
            st.rerun()
```

Project implementation details - Frontend

```
def main():  
    init_chat_state()  
    render_sidebar()  
  
    st.markdown("## 🧑‍💻 Financial Helper Chatbot")  
  
    render_chat_messages()  
    handle_user_input()
```

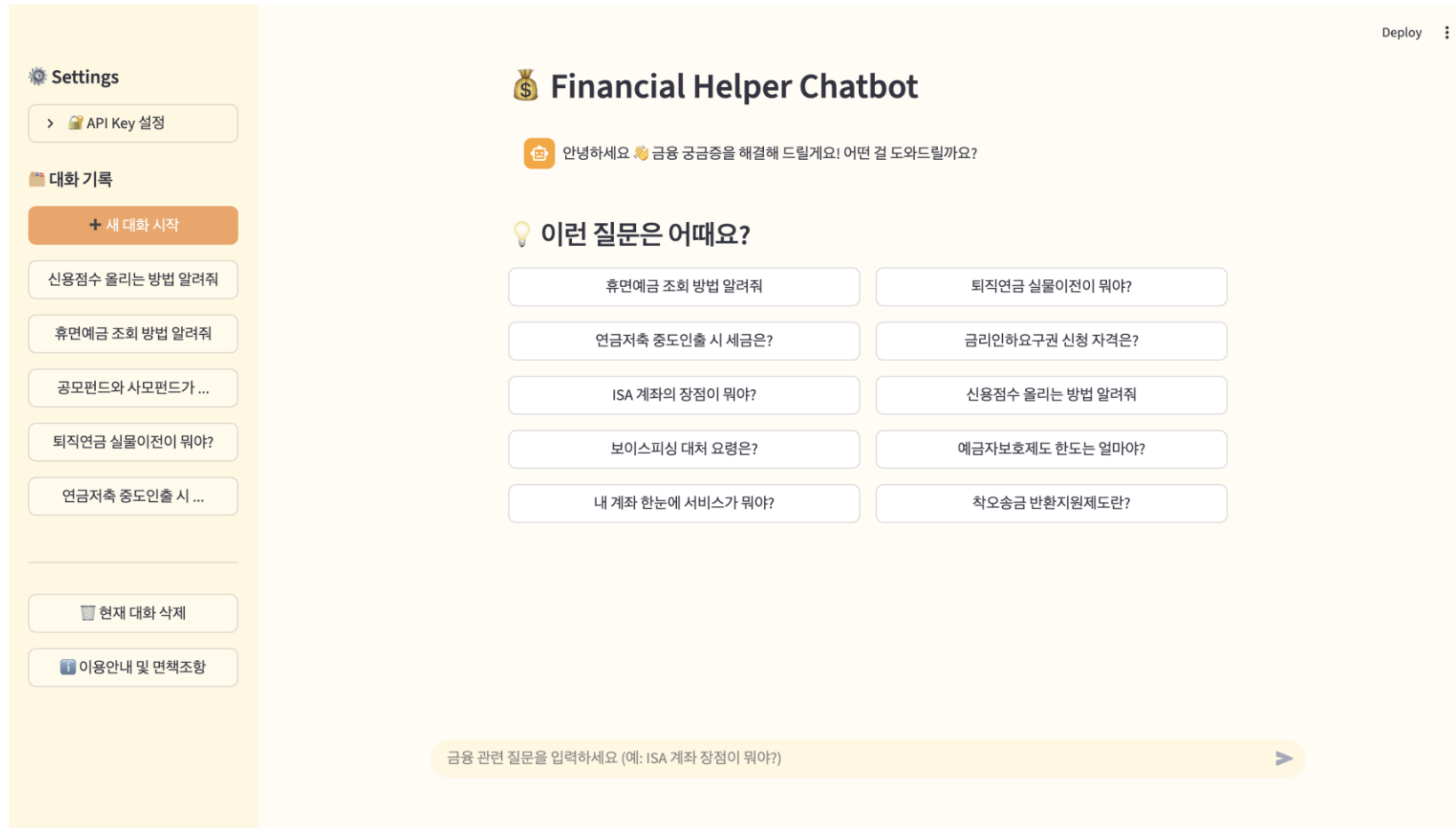
```
def render_chat_messages():  
    for msg in st.session_state.messages:  
        with st.chat_message(msg["role"]):  
            st.markdown(msg["content"])  
  
            if "sources" in msg and msg["sources"]:  
                with st.expander("참고한 자료 (출처 & 링크)"):   
                    for source in msg["sources"]:  
                        query = msg.get("query", "")  
                        display_source_item(source, search_query=query)
```

Project implementation details - Frontend

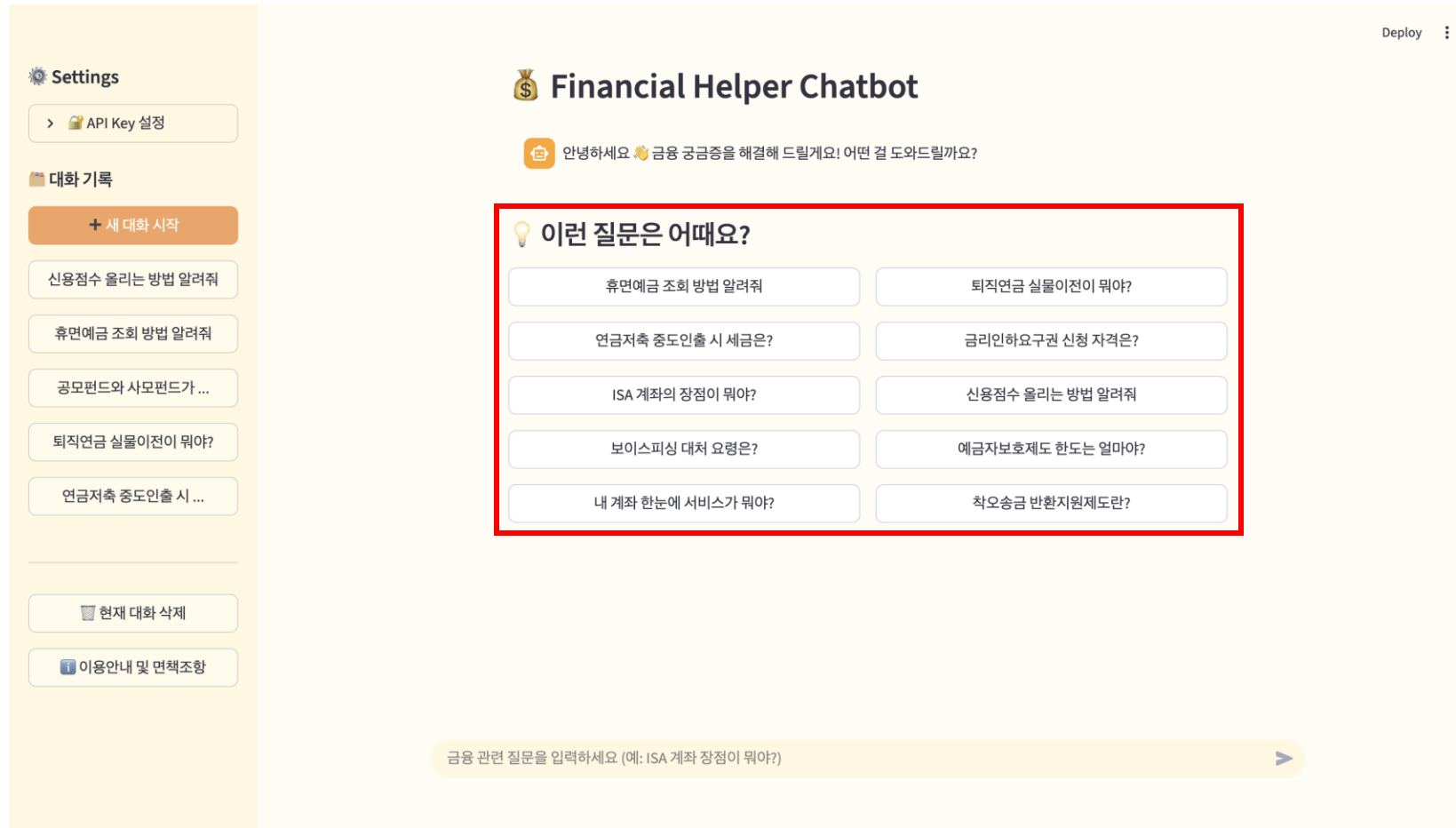
```
def main():  
    init_chat_state()  
    render_sidebar()  
  
    st.markdown("## 🧑‍💻 Financial Helper Chatbot")  
  
    render_chat_messages()  
    handle_user_input()
```

```
def handle_user_input():  
    if len(st.session_state.messages) == 1:  
        st.markdown("### 💡 이런 질문은 어때요?")  
  
    recommendations = [  
        "휴면예금 조회 방법 알려줘",  
        "퇴직연금 실물이전이 뭐야?",  
        "연금저축 중도인출 시 세금은?",  
        "금리인하요구권 신청 자격은?",  
        "ISA 계좌의 장점이 뭐야?",  
        "신용점수 올리는 방법 알려줘",  
        "보이스피싱 대처 요령은?",  
        "예금자보호제도 한도는 얼마야?",  
        "내 계좌 한눈에 서비스가 뭐야?",  
        "착오송금 반환지원제도란?"  
    ]  
  
    cols = st.columns(2)  
  
    for i, question in enumerate(recommendations):  
        if cols[i % 2].button(question, use_container_width=True):  
            process_response(question)  
            st.rerun()  
  
    user_input = st.chat_input("금융 관련 질문을 입력하세요 (예: ISA 계좌 장점이 뭐야?)")  
  
    if user_input:  
        process_response(user_input)  
        st.rerun()
```

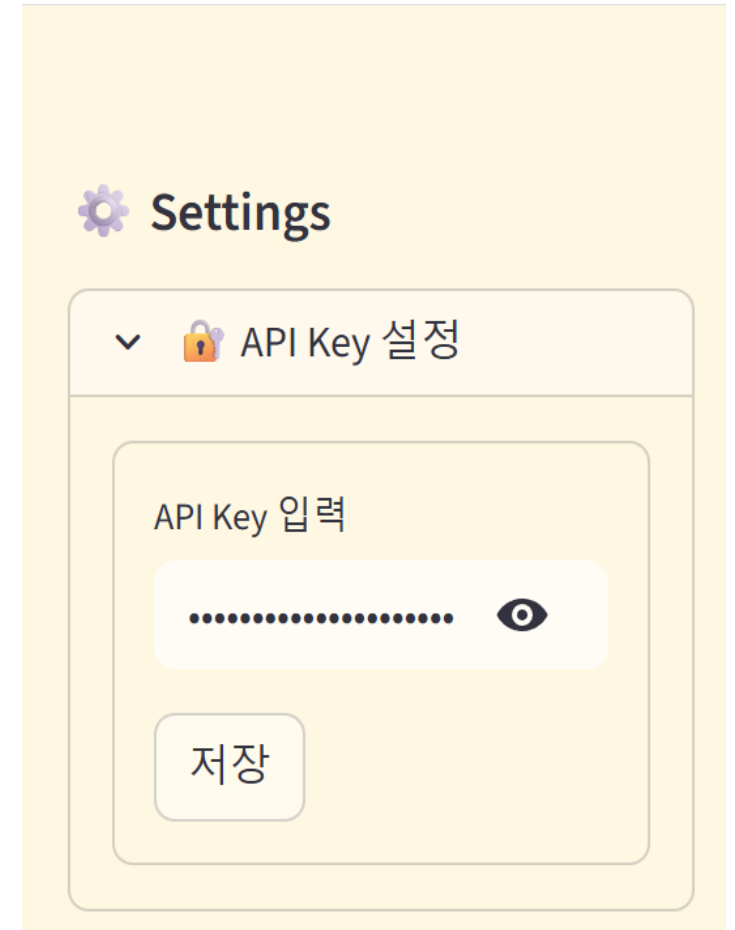
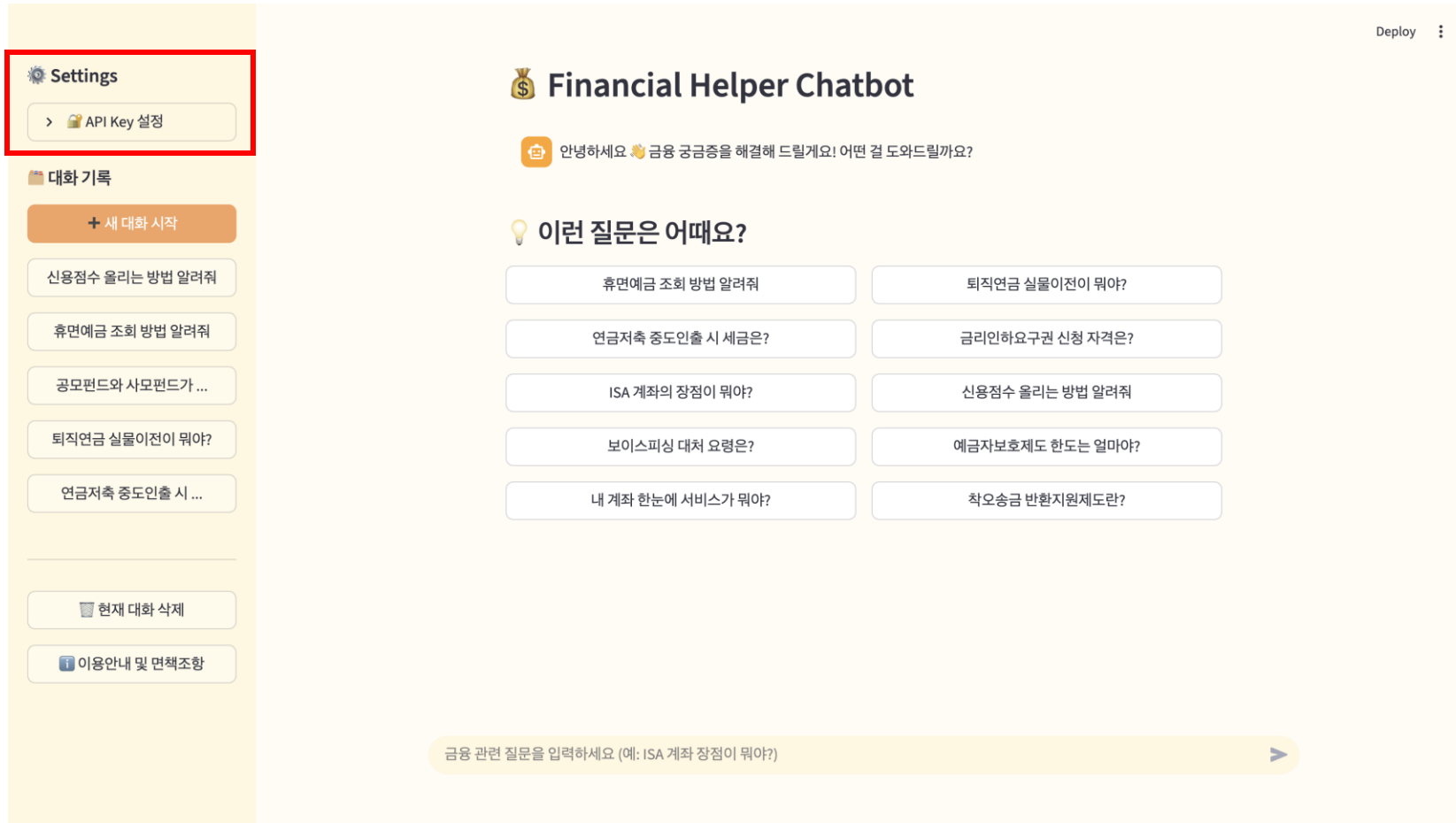
Project implementation details - Frontend



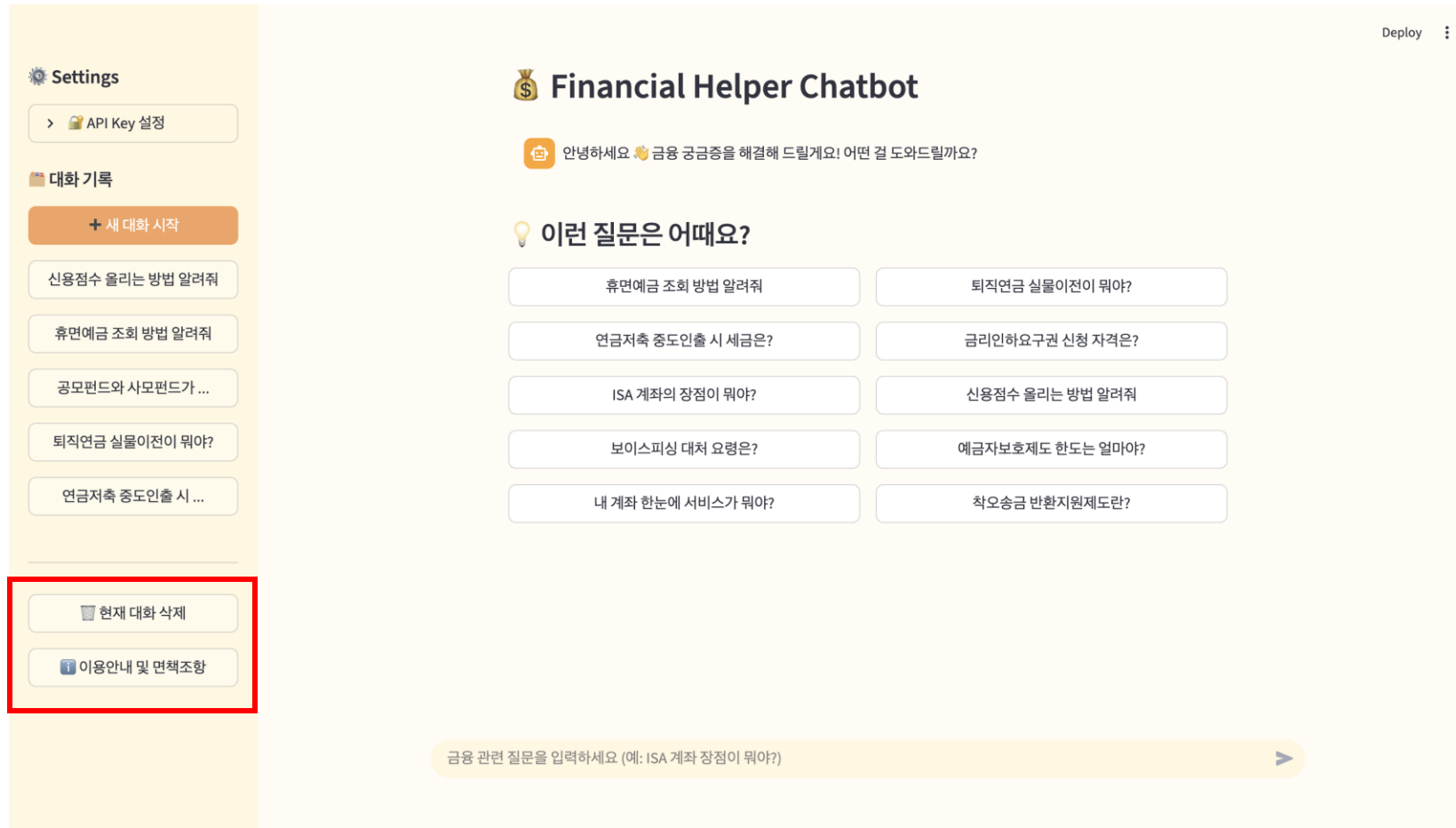
Project implementation details - Frontend



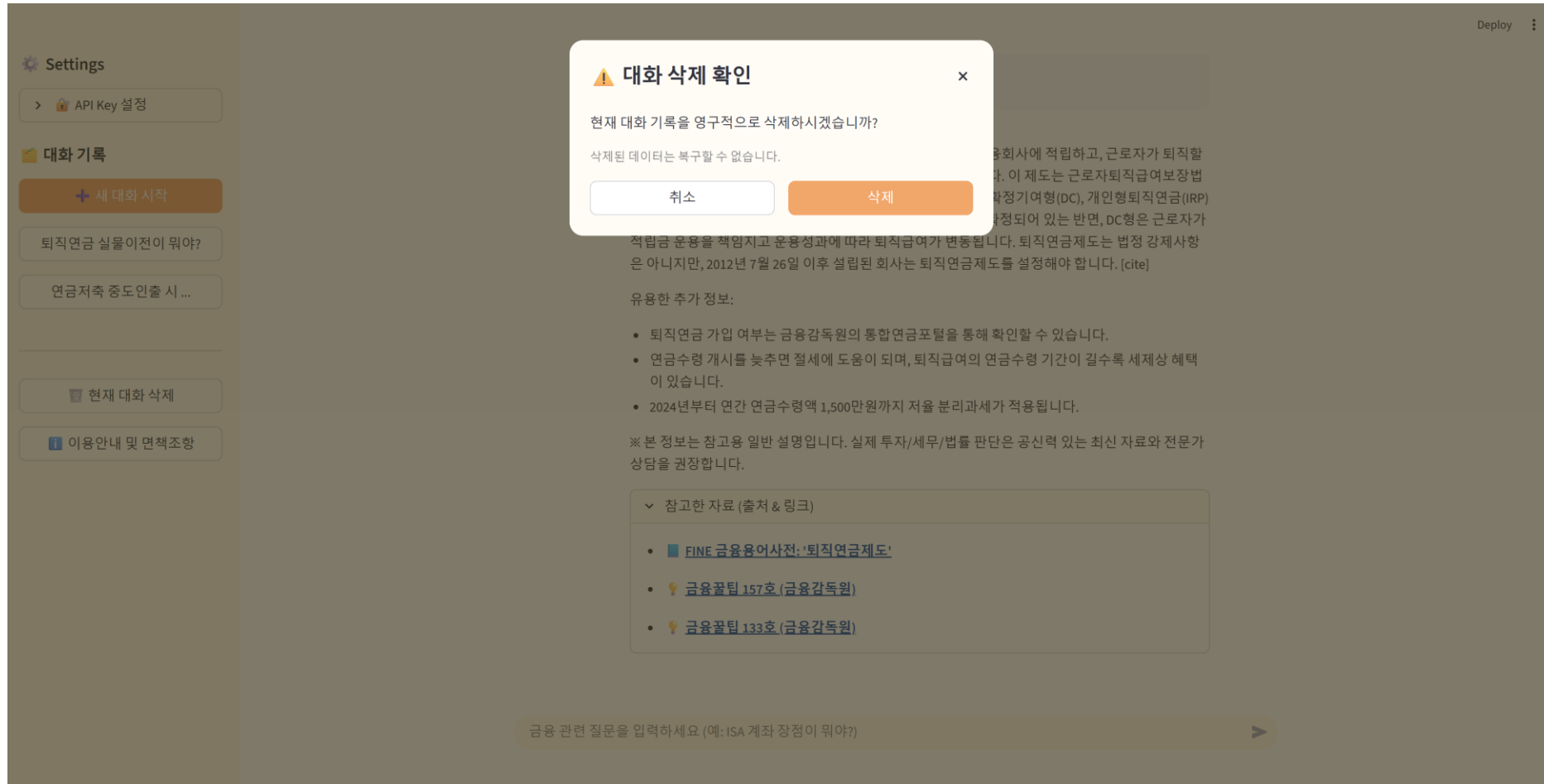
Project implementation details - Frontend



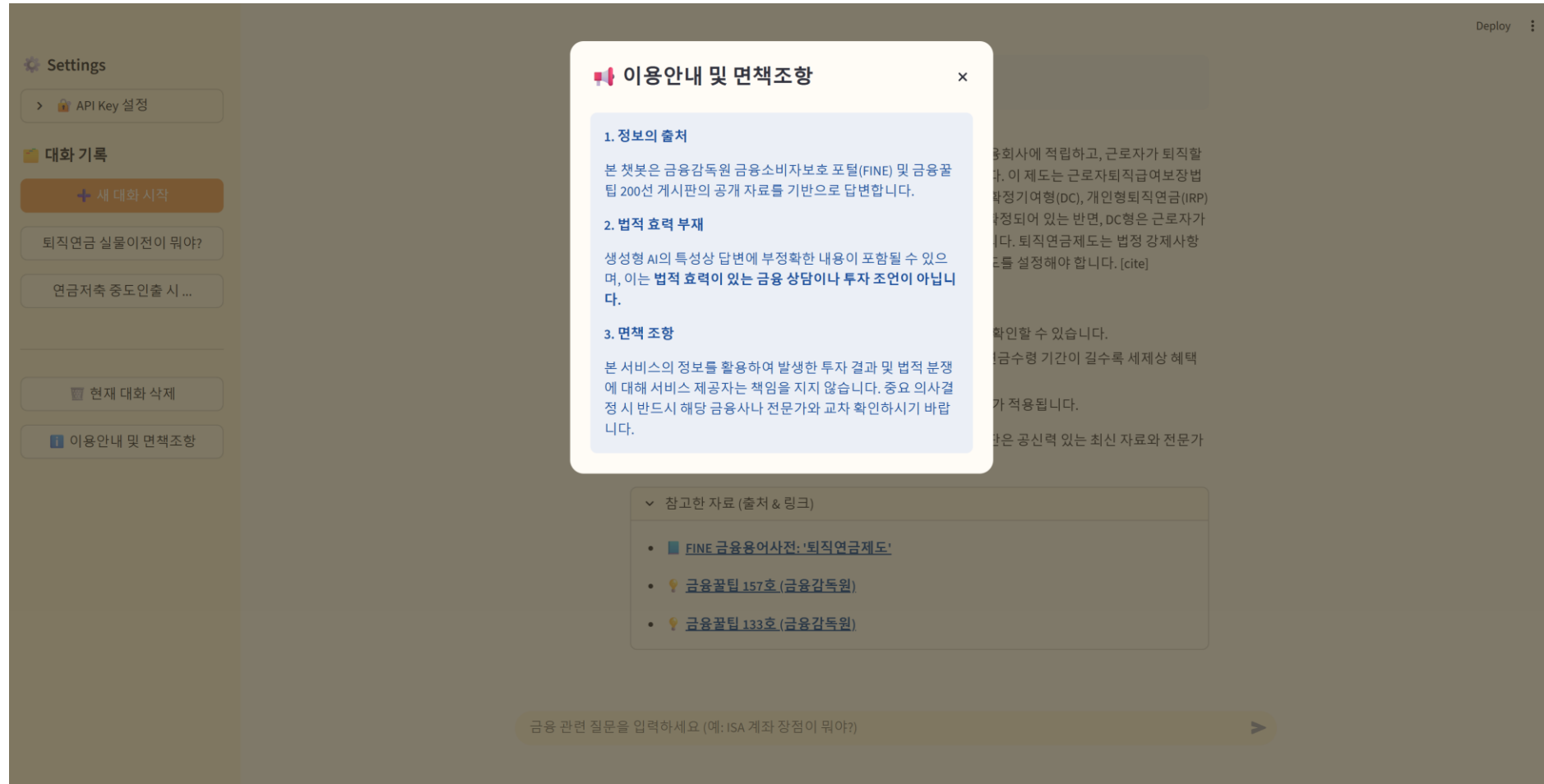
Project implementation details - Frontend



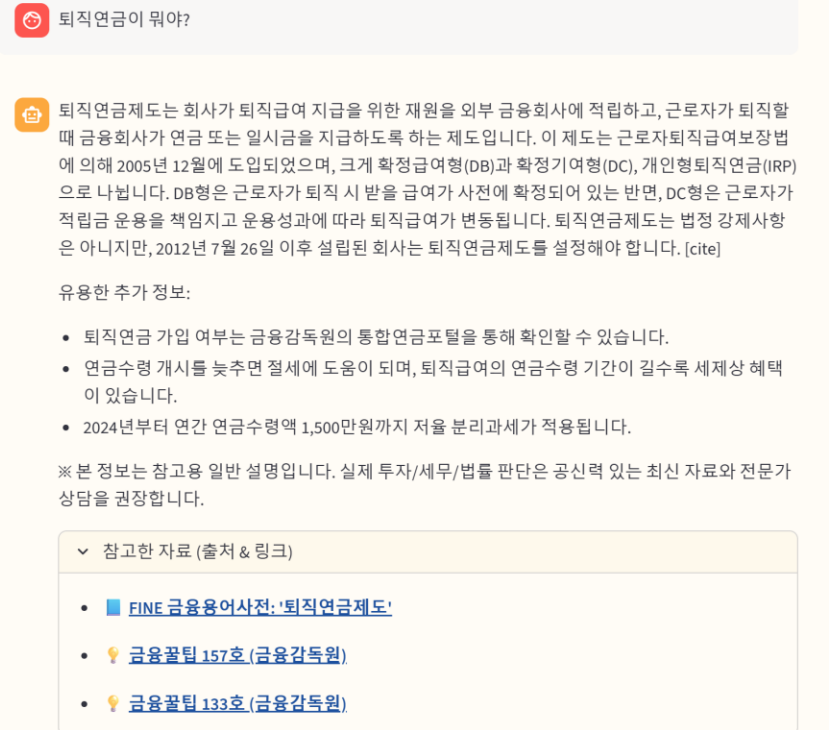
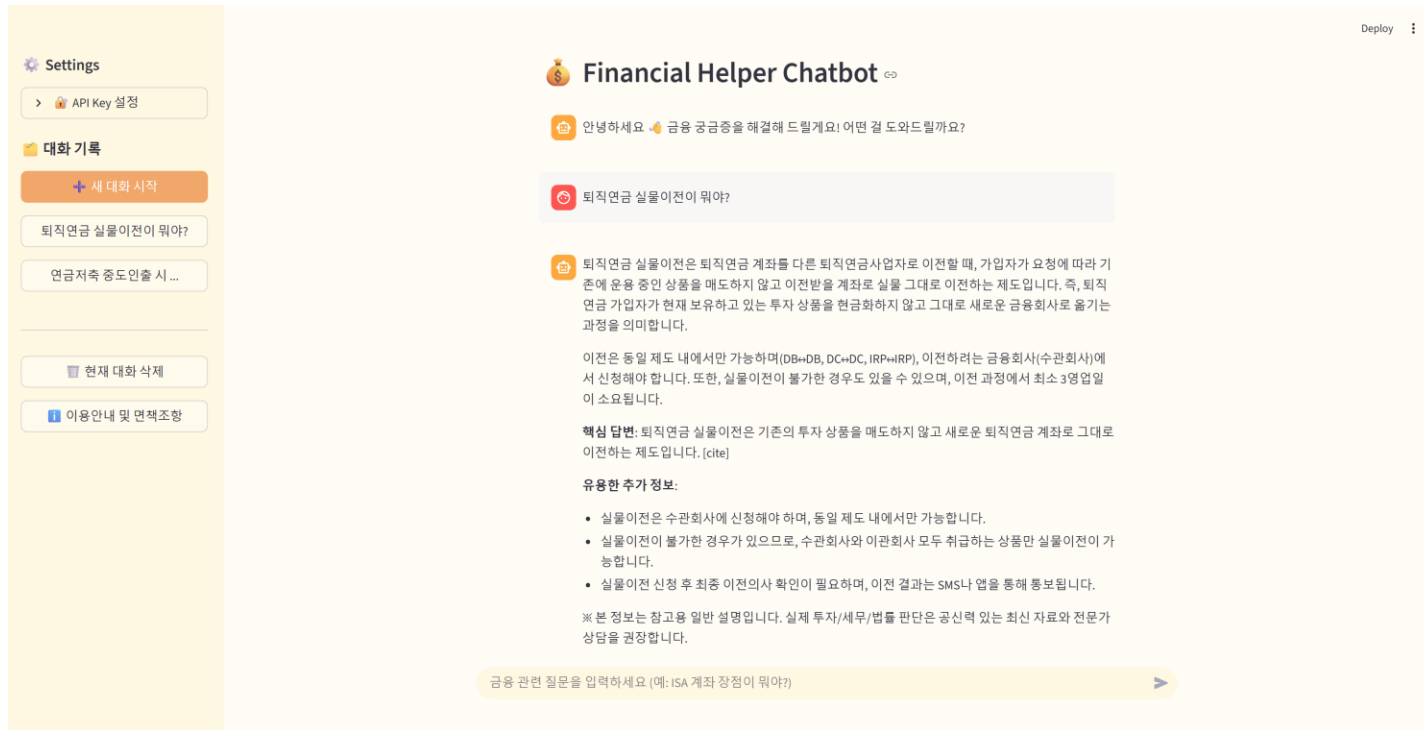
Project implementation details - Frontend



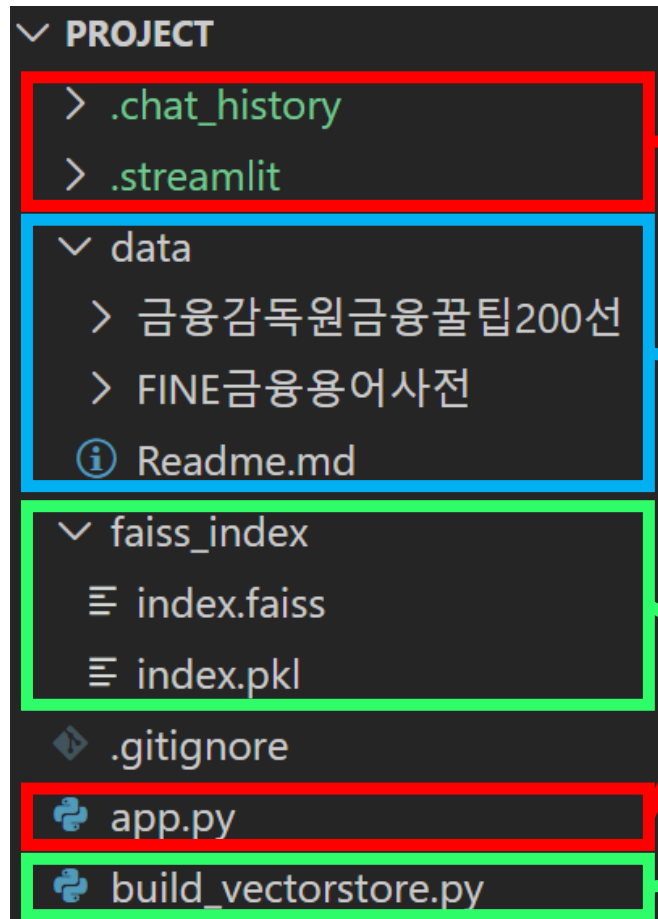
Project implementation details - Frontend



Project implementation details - Frontend



Project implementation details - Backend



Streamlit & Langchain
- Main Chatbot Implementation

Original Data
- Financial Tips
- Financial Word Dictionary

Vectorstore (FAISS)
- Faster Document Searching

Project implementation details - Backend

1. Langchain model & prompt

```
def get_rag_chain(api_key):  
    llm = get_llm(api_key, "gpt-4o-mini")  
    vectorstore = load_vectorstore(api_key)  
  
    if not vectorstore:  
        return None  
  
    retriever = vectorstore.as_retriever()  
  
    system_prompt = (  
        "너는 금융 정보를 설명하는 전문 도우미야. 너의 답변은 다음 규칙을 따라야 해:\n\n"  
        "--- 답변 규칙 ---\n"  
        "1. **(정의 검색)**: 먼저 '참고 자료'({context})에서 질문받은 용어의 **사전적 정의**가 있는지(주로 'a * tyt' 파일) 확인해.\n"  
        "2. **(Case 1: 정의 있음)**: 만약 '참고 자료'에 사전적 정의가 있다면:\n"  
        "   가. 해당 정의를 바탕으로 **핵심 답변**을 해. [cite]를 꼭 달아줘.\n"  
        "   나. '참고 자료'에 추가적인 **유용한 정보** (꿀팁, 예시 등, 주로 'c *.pdf' 파일)가 있다면 '유용한 추가 정보:' 섹션을 만들어 요약해줘.\n"  
        "3. **(Case 2: 정의 없음)**: 만약 '참고 자료'에 사전적 정의가 **없다면** (예: '적금'):\n"  
        "   가. 너의 **일반 지식**을 사용해 해당 용어의 정의를 설명해.\n"  
        "   나. **하지만**, '참고 자료'에 그 용어와 관련된 **유용한 정보** (꿀팁, 예시 등)가 검색되었다면, '참고 자료에서 찾은 유용한 정보:' 섹션을 만들어 반드시 요약해줘.\n"  
        "4. **(무관한 자료)**: 만약 '참고 자료'가 질문과 **전혀 무관**하다면, 무시하고 너의 일반 지식으로만 답해.\n"  
        "5. **(필수 안내)**: 모든 답변 마지막에는 항상 아래의 안내 문구를 추가해.\n"  
        "   * 본 정보는 참고용 일반 설명입니다. 실제 투자/세무/법률 판단은 공신력 있는 최신 자료와 전문가 상담을 권장합니다.'\n"  
        "--- 참고 자료 ---\n"  
        "{context}"  
        "---"  
    )  
  
    prompt = ChatPromptTemplate.from_messages([  
        ("system", system_prompt),  
        ("human", "{input}"),  
    ])  
  
    document_chain = create_stuff_documents_chain(llm, prompt)  
    retrieval_chain = create_retrieval_chain(retriever, document_chain)  
    return retrieval_chain
```

Use gpt-4o-mini model

If there is a word in dictionary, show definition first.

If there does not exist, just show related tips.

Make chain

Project implementation details - Backend

2. Chat history – Pickle

```
def save_chat_history():
    """현재 채팅 기록을 로컬 파일(pickle)로 저장 (제목 포함)"""
    session_id = get_session_id()
    file_path = os.path.join(HISTORY_DIR, f"chat_{session_id}.pkl")

    title = "새로운 대화"
    for msg in st.session_state.messages:
        if msg["role"] == "user":
            if len(msg["content"]) > 15:
                title = msg["content"][:12] + "..."
            else:
                title = msg["content"][:15]
            break

    data = {
        "title": title,
        "messages": st.session_state.messages
    }

    try:
        with open(file_path, "wb") as f:
            pickle.dump(data, f)
    except Exception as e:
        st.error(f"대화 저장 중 오류 발생: {e}")
```

Save & Load Chat history by using pickle

```
def load_chat_history(filename):
    file_path = os.path.join(HISTORY_DIR, filename)
    try:
        with open(file_path, "rb") as f:
            data = pickle.load(f)

        if isinstance(data, list):
            st.session_state.messages = data
        else:
            st.session_state.messages = data["messages"]

        session_id = filename.replace("chat_", "").replace(".pkl", "")
        st.session_state.session_id = session_id

        return True
    except Exception as e:
        st.error(f"대화 불러오기 실패: {e}")
        return False
```

Project implementation details - Backend

3. Vectorstore - FAISS

```
@st.cache_resource(show_spinner="[1/2] LLM 모델을 준비하는 중...")
def get_llm(api_key: str, model: str):
    return ChatOpenAI(api_key=api_key, model=model, temperature=0.2)

@st.cache_resource(show_spinner="[2/2] 저장된 금융 문서를 로드하는 중...")
def load_vectorstore(api_key: str):
    VECTORSTORE_PATH = "faiss_index"
    embeddings = OpenAIEmbeddings(api_key=api_key)
    try:
        vectorstore = FAISS.load_local(
            VECTORSTORE_PATH,
            embeddings,
            allow_dangerous_deserialization=True
        )
        return vectorstore
    except Exception as e:
        st.error(f"벡터 스토어 로드 실패: {e}")
        return None
```

Load Vectorstore

```
def build_vectorstore():
    import streamlit as st
    print("벡터 스토어 빌드를 시작합니다...")

    # 1. API 키 확인
    api_key = st.secrets.get("openai_api_key", None)
    # 2) if not exist, find OPENAI_API_KEY
    if not api_key:
        api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        print("오류: OPENAI_API_KEY 환경 변수가 설정되지 않았습니다.")
        print(".env 파일을 생성하거나 터미널에서 직접 키를 설정해주세요.")
        return

    # 2. 문서 경로 정의
    pdf_path = "./data/금융감독원금융포털200선"
    txt_path = "./data/FINE금융용어사전"

    # 3. 로더 생성 및 문서 로드
    print(f"{pdf_path}에서 PDF 문서를 로드합니다...")
    pdf_loader = PyPDFDirectoryLoader(pdf_path)
    pdf_docs = pdf_loader.load()

    print(f"{txt_path}에서 TXT 문서를 로드합니다...")
    txt_loader = DirectoryLoader(
        txt_path,
        glob="**/*.txt",
        loader_cls=TextLoader,
        loader_kwargs={"encoding": "utf-8"}
    )
    txt_docs = txt_loader.load()
    all_docs = pdf_docs + txt_docs
    print(f"총 {len(all_docs)}개의 문서를 로드했습니다.")

    # 4. 문서 분할 (Chunking)
    print("문서를 분할합니다 (Chunking)...")
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
    splits = text_splitter.split_documents(all_docs)
    print(f"총 {len(splits)}개의 문서 조각으로 분할되었습니다.")

    # 5. 임베딩 모델 생성
    print("OpenAI 임베딩 모델을 사용해 벡터화를 시작합니다... (시간이 걸립니다)")
    embeddings = OpenAIEmbeddings(api_key=api_key, chunk_size=200)

    # 6. 벡터 스토어 생성 (FAISS) 및 저장
    start_time = time.time()
    vectorstore = FAISS.from_documents(splits, embeddings)
    end_time = time.time()

    print(f"벡터화 완료. (소요 시간: {end_time - start_time:.2f}초)")

    # 7. 로컬에 저장
    vectorstore.save_local(VECTORSTORE_PATH)
    print(f"성공: 벡터 스토어를 '{VECTORSTORE_PATH}' 폴더에 저장했습니다.")
```

Load
Embedding Model

Load Documents
(Tips, Dictionary)

Chunking &
Embedding

Save Vectorstore

Demonstration

Video

